

Teach Module 7: Exception Handling and Error Management

Module 7 covers exception handling and error management in Java.

Topics:

- Checked versus unchecked exceptions
- `try`, `catch`, and `finally`
- Custom exception design
- Error propagation strategies
- Logging exceptions for diagnostics
- Practice lab

Lesson

In Java, an exception is Java's way of saying:

Something went wrong while the program was running, and normal execution cannot continue safely.

Examples:

```
int result = 10 / 0; // ArithmeticException
```

```
String name = null;  
System.out.println(name.length()); // NullPointerException
```

```
FileReader reader = new FileReader("missing.txt"); // FileNotFoundException
```

Exceptions help us avoid programs crashing blindly. Instead, we can detect the problem, respond properly, and give useful feedback.

Checked Vs Unchecked Exceptions

Checked exceptions are exceptions Java forces you to handle at compile time.

Example:

```
import java.io.FileReader;  
import java.io.FileNotFoundException;  
  
public class App {  
    public static void main(String[] args) throws FileNotFoundException {  
        FileReader reader = new FileReader("data.txt");  
    }  
}
```

`FileNotFoundException` is checked because Java says this operation might fail, so the method must either handle it or declare that it throws it.

Common checked exceptions:

```
IOException  
FileNotFoundException  
SQLException  
ClassNotFoundException
```

Use checked exceptions when the caller can reasonably recover.

Unchecked exceptions are not forced by the compiler. They usually mean a programming mistake or invalid method usage.

Example:

```
public class App {  
    public static void main(String[] args) {  
        int[] numbers = {1, 2, 3};  
        System.out.println(numbers[5]);  
    }  
}
```

This causes:

```
ArrayIndexOutOfBoundsException
```

Common unchecked exceptions:

```
NullPointerException  
IllegalArgumentException  
IllegalStateException  
ArithmeticException  
IndexOutOfBoundsException
```

Unchecked exceptions extend `RuntimeException`.

Example:

```
public void setAge(int age) {  
    if (age < 0) {  
        throw new IllegalArgumentException("Age cannot be negative");  
    }  
}
```

Simple rule:

```
Checked exception: external problem the program might recover from.  
Unchecked exception: programming error or invalid method usage.
```

Try, Catch, And Finally

The basic structure:

```
try {  
    // risky code  
} catch (ExceptionType e) {  
    // handle the problem  
} finally {  
    // always runs  
}
```

Example:

```
public class App {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0;  
            System.out.println(result);  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero.");  
        } finally {  
            System.out.println("Cleanup complete.");  
        }  
    }  
}
```

Output:

```
Cannot divide by zero.  
Cleanup complete.
```

The `finally` block runs whether an exception happens or not.

Typical uses for `finally` :

```
Close files  
Close database connections  
Release resources  
Clean up temporary state
```

Modern Java often uses `try-with-resources` instead of manual `finally` .

Example:

```
try (FileReader reader = new FileReader("data.txt")) {  
    // read file  
} catch (IOException e) {
```

```
        System.out.println("Could not read file.");
    }
```

Java automatically closes the resource.

Multiple Catch Blocks

You can catch different exceptions differently:

```
try {
    String input = "abc";
    int number = Integer.parseInt(input);
    System.out.println(10 / number);
} catch (NumberFormatException e) {
    System.out.println("Input must be a number.");
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero.");
}
```

Order matters. Catch specific exceptions first.

Bad:

```
catch (Exception e) {
}
catch (NumberFormatException e) {
}
```

Good:

```
catch (NumberFormatException e) {
}
catch (Exception e) {
}
```

Throwing Exceptions

You can throw your own exception when something invalid happens.

```
public class BankAccount {
    private double balance;

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Amount must be positive");
        }

        if (amount > balance) {
            throw new IllegalStateException("Insufficient funds");
        }
    }
}
```

```
    }

    balance -= amount;
}
}
```

`IllegalArgumentException` means the caller passed a bad value.

`IllegalStateException` means the object is not in the right state for that operation.

Custom Exceptions

Sometimes built-in exceptions are not expressive enough.

Example:

```
public class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String message) {
        super(message);
    }
}
```

Use it:

```
public class BankAccount {
    private double balance;

    public void withdraw(double amount) {
        if (amount > balance) {
            throw new InsufficientFundsException("Not enough balance to withdraw " +
amount);
        }

        balance -= amount;
    }
}
```

Custom exceptions make your domain clearer.

Instead of this:

```
throw new RuntimeException("Payment failed");
```

Prefer this:

```
throw new PaymentFailedException("Card was declined");
```

A good custom exception name explains the business problem.

Error Propagation

Error propagation means deciding where an exception should be handled.

Example:

```
public User findUserId(int id) {  
    if (id <= 0) {  
        throw new IllegalArgumentException("User ID must be positive");  
    }  
  
    return userRepository.findById(id);  
}
```

Lower-level methods often throw exceptions upward. Higher-level methods decide what to do.

Example flow:

```
Repository throws DatabaseException  
Service catches or transforms it  
Controller returns a useful API response
```

In backend Java apps, a common pattern is:

```
throw new UserNotFoundException("User not found with id: " + id);
```

Then the API layer converts it into:

```
HTTP 404 Not Found
```

Logging Exceptions

Never silently swallow exceptions.

Bad:

```
try {  
    processPayment();  
} catch (Exception e) {  
}
```

Better:

```
try {  
    processPayment();  
} catch (PaymentException e) {  
    logger.error("Payment processing failed", e);  
}
```

A good log message explains what failed. Passing the exception object preserves the stack trace.

Avoid this:

```
logger.error(e.getMessage());
```

Better:

```
logger.error("Unable to process payment for order {}", orderId, e);
```

Practice Exercises

Exercise 1: Safe Division

Create a method:

```
public int divide(int a, int b)
```

Requirements:

```
If b is 0, throw IllegalArgumentException.  
Otherwise return a / b.  
Call the method inside try/catch.  
Print a friendly error message if division fails.
```

Exercise 2: Age Validator

Create:

```
public void validateAge(int age)
```

Requirements:

```
If age is less than 0, throw IllegalArgumentException.  
If age is less than 18, throw custom exception UnderageException.  
Otherwise print "Access granted."
```

Exercise 3: File Reader With Try-With-Resources

Create a program that reads a file called:

```
users.txt
```

Requirements:

```
Use try-with-resources.  
Catch IOException.
```

Print "Unable to read users file" if reading fails.
Do not crash the program.

Exercise 4: Bank Withdrawal

Create a `BankAccount` class.

```
public void withdraw(double amount)
```

Requirements:

If amount is less than or equal to 0, throw `IllegalArgumentException`.
If amount is greater than balance, throw `InsufficientFundsException`.
Otherwise subtract amount from balance.

Exercise 5: Order Processing

Create:

```
public void placeOrder(String productId, int quantity)
```

Requirements:

If `productId` is null or blank, throw `IllegalArgumentException`.
If quantity is less than or equal to 0, throw `IllegalArgumentException`.
If quantity is greater than 100, throw `BulkOrderLimitExceededException`.
Otherwise print "Order placed successfully."

Exercise 6: Login System

Create:

```
public void login(String username, String password)
```

Requirements:

If username is null or blank, throw `IllegalArgumentException`.
If password is null or blank, throw `IllegalArgumentException`.
If username is not "admin" or password is not "password123", throw `InvalidLoginException`.
Otherwise print "Login successful."

Exercise 7: Exception Propagation

Create three classes:

```
UserController  
UserService  
UserRepository
```


Requirements:

```
UserRepository.findById(int id) throws UserNotFoundException if id is not found.  
UserService calls UserRepository.  
UserController catches UserNotFoundException and prints "404: User not found."
```

Exercise 8: Logging Simulation

Create a method:

```
public void processPayment(String orderId)
```

Requirements:

```
Throw PaymentFailedException if orderId is null or blank.  
Catch the exception in main.  
Print a diagnostic message that includes the orderId.  
Print the exception message.
```

Example output:

```
ERROR: Payment failed for order ORD-101  
Reason: Invalid order ID
```

Exercise 9: Multiple Catch Blocks

Create a program that accepts a string number.

```
String input = "abc";
```

Requirements:

```
Convert input to int.  
Divide 100 by that number.  
Catch NumberFormatException separately.  
Catch ArithmeticException separately.  
Catch general Exception last.
```

Exercise 10: Student Registration Mini Project

Build a small Student Registration System.

Classes:

```
Student  
StudentService  
InvalidStudentException  
DuplicateStudentException
```

```
StudentNotFoundException  
Main
```

Requirements:

```
Student name cannot be null or blank.  
Student age must be 16 or older.  
Student email must contain "@"  
Duplicate student emails are not allowed.  
Searching for a missing student throws StudentNotFoundException.  
Main should catch exceptions and print friendly messages.
```

Recommended practice order:

1. Safe Division
2. Age Validator
3. Bank Withdrawal
4. File Reader
5. Order Processing
6. Exception Propagation
7. Student Registration Mini Project

Module 7 Lab: Order Processing Lab

Build a small Java console app called Order Processing Lab.

Goal:

```
Checked vs unchecked exceptions  
try/catch/finally  
Custom exceptions  
Exception propagation  
Diagnostic error messages
```

Scenario:

You are building a simple order service. A customer can place an order for a product, but the system must reject invalid orders safely.

Create these classes:

```
Main  
Order  
OrderService  
InvalidOrderException  
ProductNotFoundException  
BulkOrderLimitExceededException
```

Step 1: Create The Order Class

Fields:

```
private String productId;  
private int quantity;  
private double price;
```

Add:

```
Constructor  
Getters  
toString()
```

Step 2: Create Custom Exceptions

Create three custom exceptions:

```
public class InvalidOrderException extends RuntimeException {  
    public InvalidOrderException(String message) {  
        super(message);  
    }  
}
```

```
public class ProductNotFoundException extends RuntimeException {  
    public ProductNotFoundException(String message) {  
        super(message);  
    }  
}
```

```
public class BulkOrderLimitExceededException extends RuntimeException {  
    public BulkOrderLimitExceededException(String message) {  
        super(message);  
    }  
}
```

Step 3: Create OrderService

Add this method:

```
public void placeOrder(Order order)
```

Validation rules:

```
If order is null, throw InvalidOrderException.  
If productId is null or blank, throw InvalidOrderException.  
If quantity is less than or equal to 0, throw InvalidOrderException.  
If quantity is greater than 100, throw BulkOrderLimitExceededException.
```

If `productId` is not "P100", "P200", or "P300", throw `ProductNotFoundException`. Otherwise print the order total.

Example:

```
double total = order.getQuantity() * order.getPrice();
System.out.println("Order placed successfully. Total: $" + total);
```

Step 4: Handle Exceptions In Main

In `main`, create several test orders:

```
new Order("P100", 2, 25.0);    // valid
new Order("", 2, 25.0);        // invalid product ID
new Order("P999", 2, 25.0);    // product not found
new Order("P200", 150, 10.0);  // bulk limit exceeded
new Order("P300", -1, 10.0);   // invalid quantity
```

Call `placeOrder()` inside `try/catch`.

Use separate catch blocks:

```
try {
    orderService.placeOrder(order);
} catch (InvalidOrderException e) {
    System.out.println("Invalid order: " + e.getMessage());
} catch (ProductNotFoundException e) {
    System.out.println("Product error: " + e.getMessage());
} catch (BulkOrderLimitExceededException e) {
    System.out.println("Bulk order error: " + e.getMessage());
} finally {
    System.out.println("Order processing attempt finished.");
}
```

Expected behavior:

The program should not crash. It should print friendly messages for each failed order.

Example output:

```
Order placed successfully. Total: $50.0
Order processing attempt finished.

Invalid order: Product ID cannot be blank
Order processing attempt finished.

Product error: Product not found: P999
Order processing attempt finished.

Bulk order error: Cannot order more than 100 units
Order processing attempt finished.
```

```
Invalid order: Quantity must be greater than 0  
Order processing attempt finished.
```

Bonus Challenge

Add a method:

```
public void saveOrderToFile(Order order)
```

Make it throw or handle `IOException` .

This adds practice with checked exceptions.